

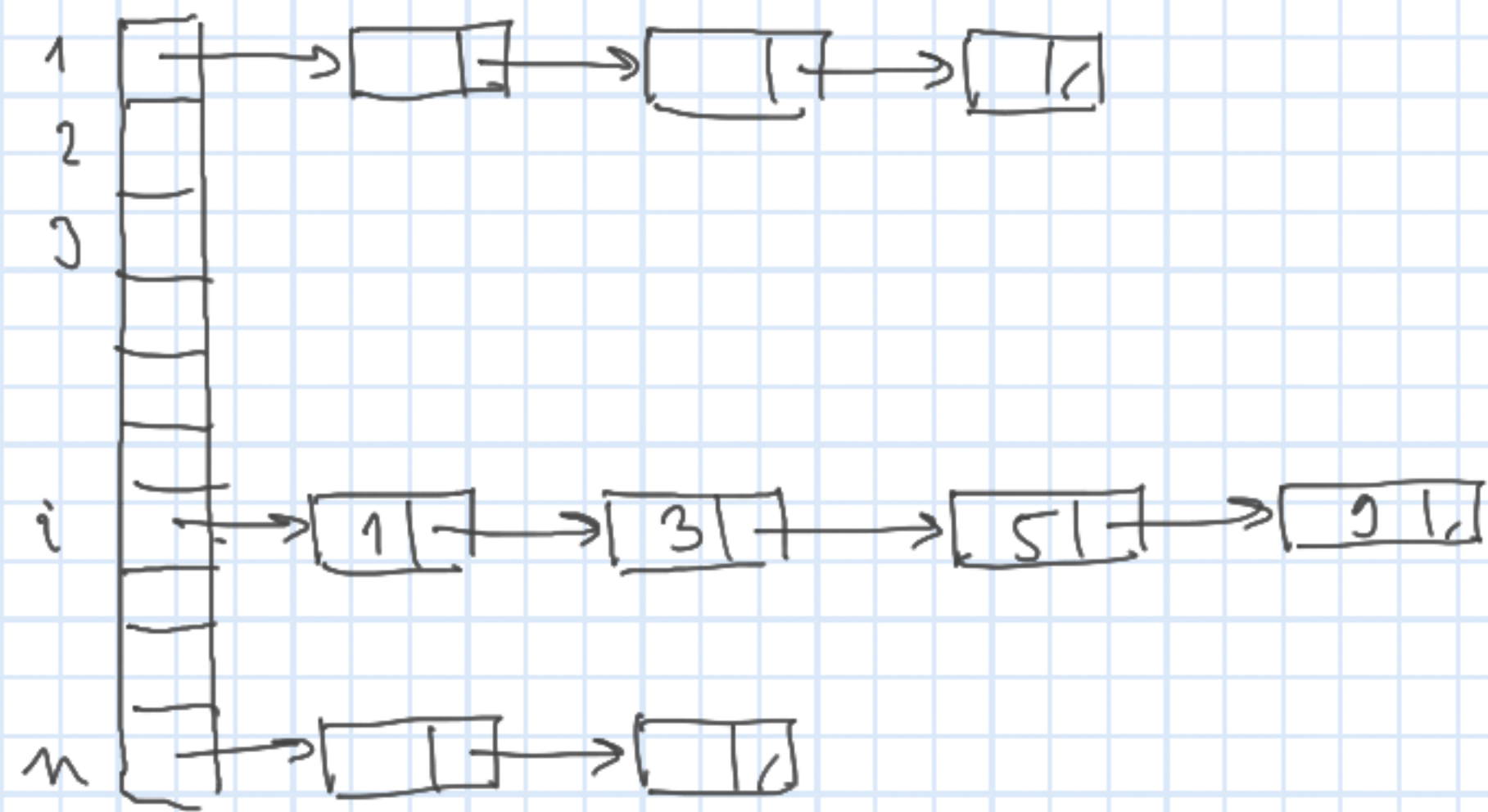
September 24, 2021

Adjacency matrix (cont'd)

If our graph is weighted the adjacency matrix contains the weights instead of 0's and 1's

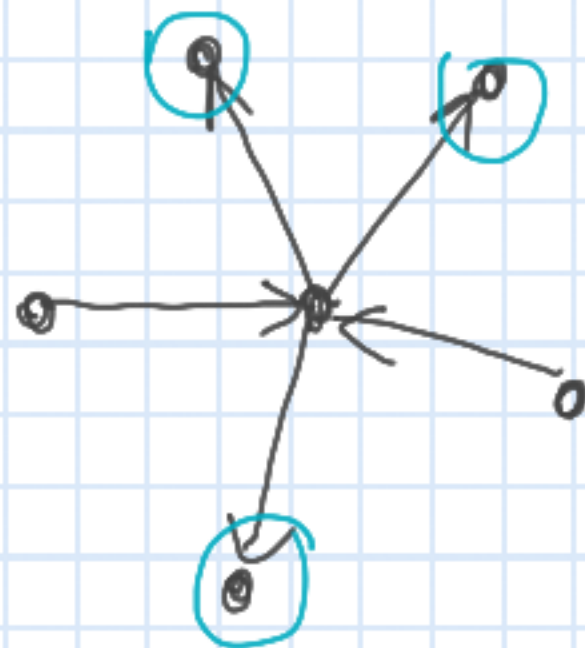
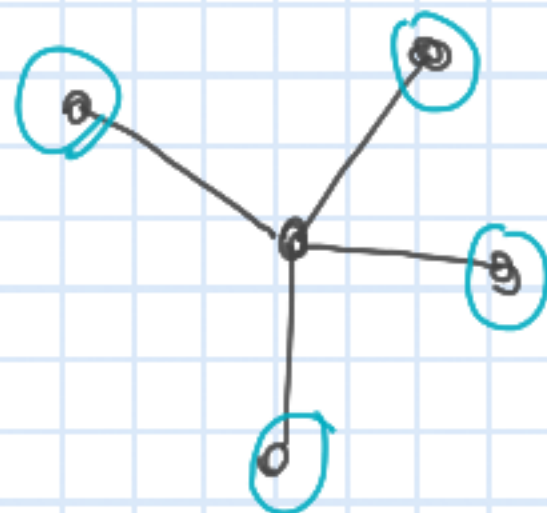
Adjacency lists

(again $V = \{1, 2, \dots, n\}$)

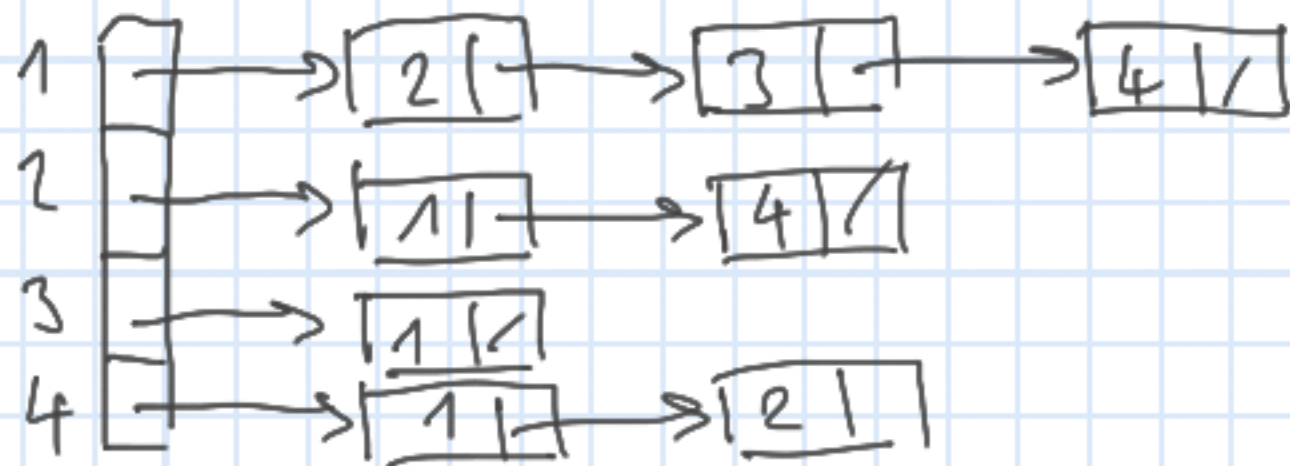
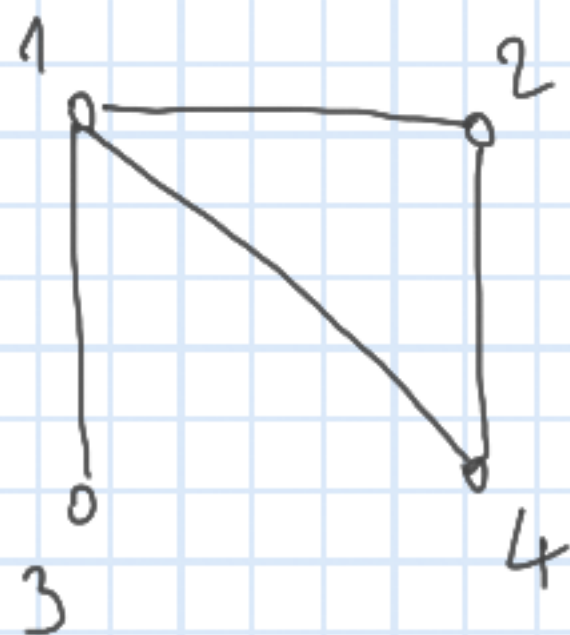


Adj

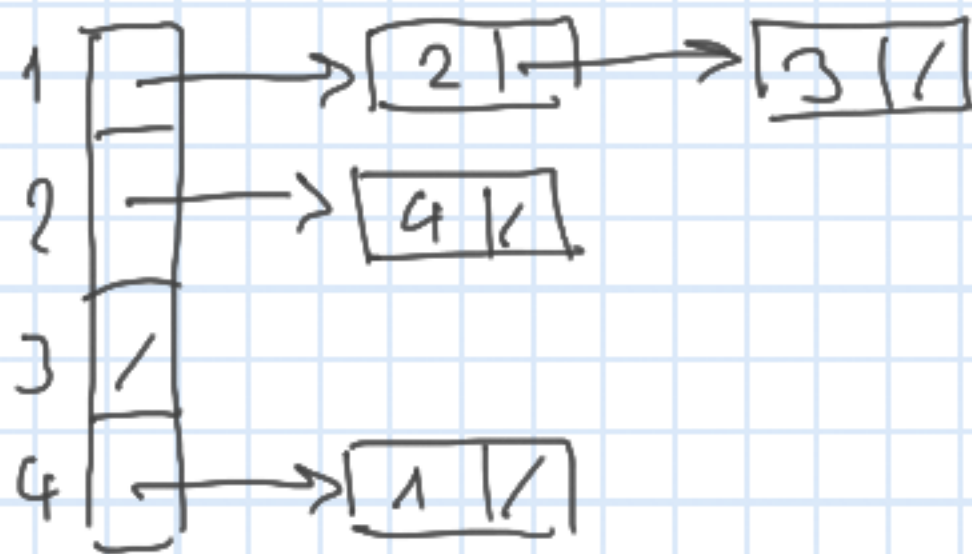
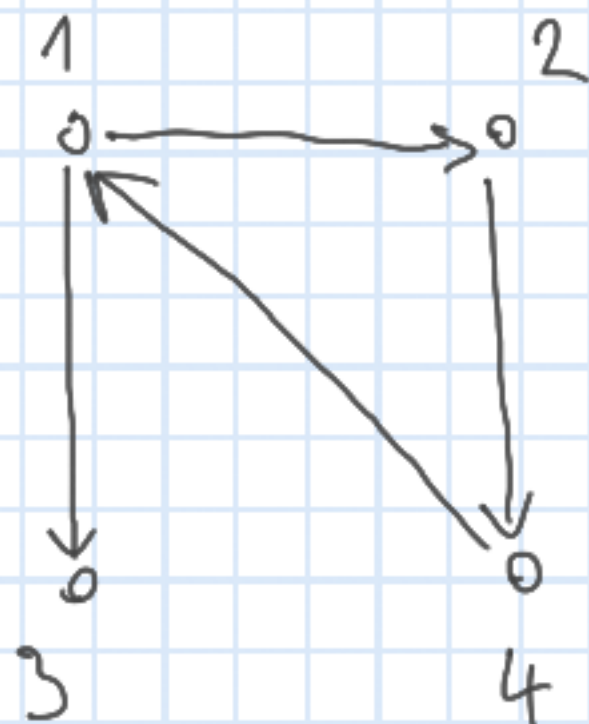
linked lists containing
the "neighbours" of the
vertices



Example



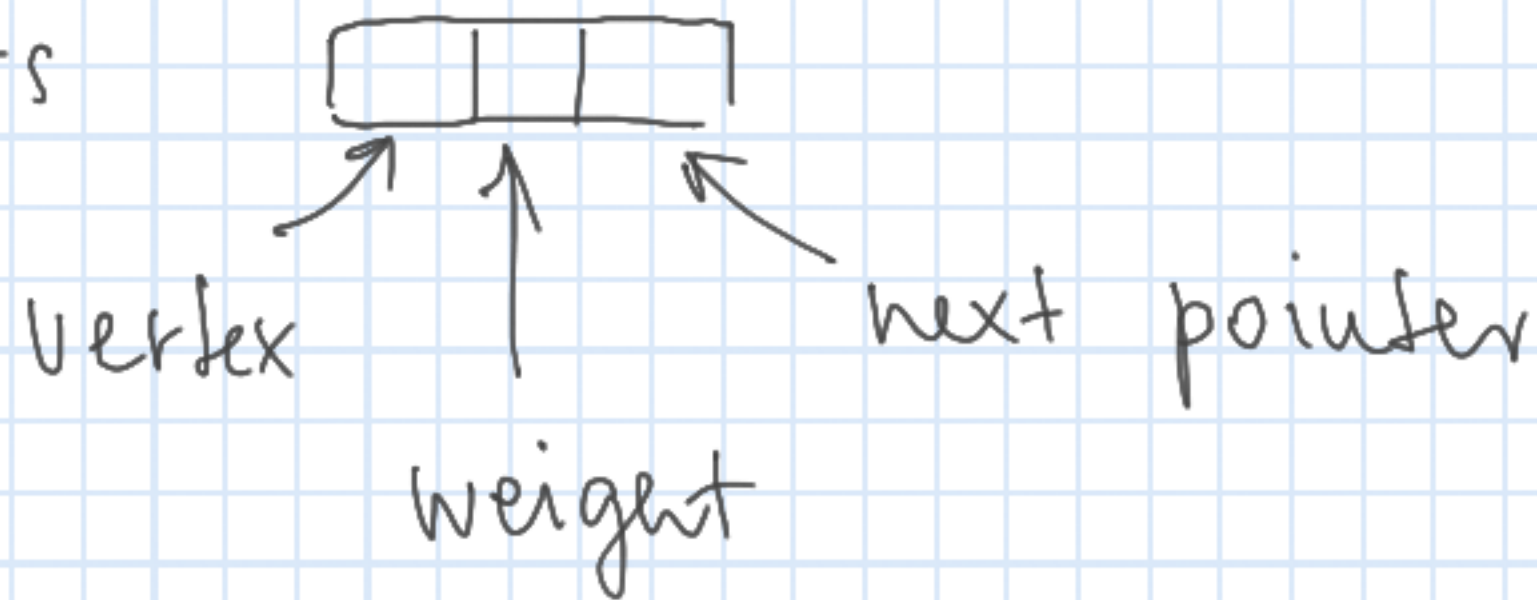
Adj



Adj

weighted graphs

list objects



Comparison of the two representations

Adjacency matrix

$$\Theta(|V|^2)$$

$$O(1)$$

Adjacency list

space compl. $\Theta(|V| + |E|)$

answering the question $(i, j) \in E?$ slower

in the worst case $\Omega(|V|)$

One important point

linear time graph algorithm $\rightarrow \Theta(|V| + |E|)$

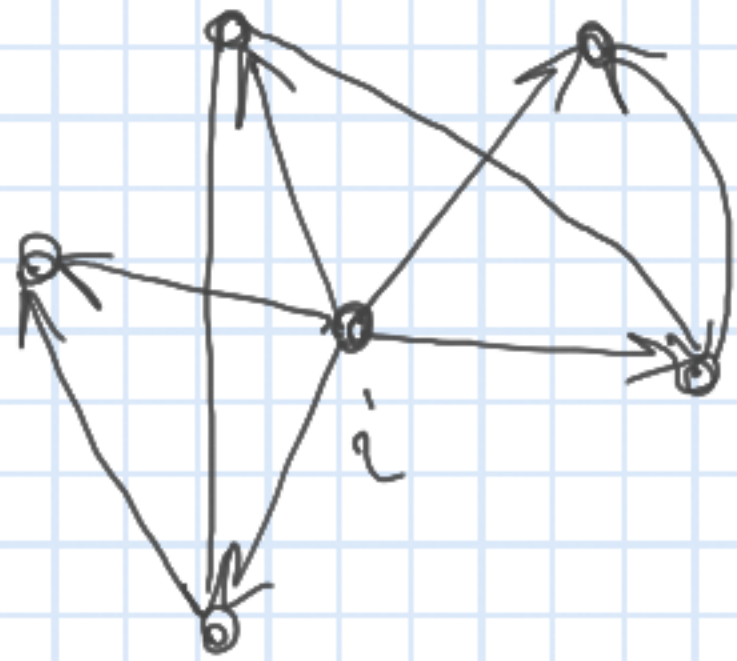
(in a linear time algorithm the graph is given by adjacency lists, esp.)

Two problems concerning adjacency matrix representations

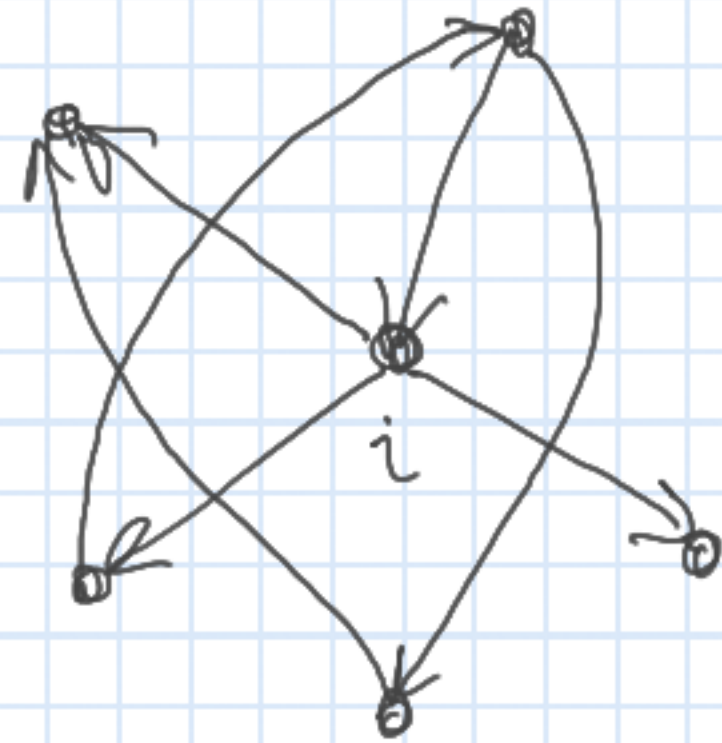
① We are given a directed graph $G = (V, E)$ by its adjacency matrix (again $V = \{1, 2, \dots, n\}$)

We say that a vertex $i \in V$ is a super-source in G if for every other vertex $j \in V \setminus \{i\}$ we have $(i, j) \in E$ but $(j, i) \notin E$.

Example



i is a supersource



i is not a s.s.
no supersource here

Design an algorithm that decide if a
s.s exists or not in G !

Note that if a supersource exists, it must be unique.

Naïv approach

We can check whether $i \in V$ is a S.S.:

$$\text{Adj} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

i^{th}

j^{th}

i h a s.s.

Do this for $i = 1, 2, \dots, n$ until we find a supersource or we recognize that no supersource exists.

Time complexity: $O(V^2)$

Is there a more efficient one? Say some $O(V)$?

② Connectivity test

We are given an undirected graph $G = (V, E)$ by its adjacency matrix and we want to decide whether G is connected or not.

Show that a general algorithm solving this problem must check all entries in the upper triangle part of the adjacency matrix in the worst case.

$$Adj = \begin{bmatrix} 0 & & & & \\ & 0 & & & \\ & & 0 & & \\ & & & 0 & \\ & & & & 0 \end{bmatrix} \leftarrow \begin{array}{l} \text{upper triangle part with} \\ \binom{n}{2} = \frac{n(n-1)}{2} \text{ items} \end{array}$$

It isn't easy!

Now we turn back to the supersource problem.

Critical observation: if $(i, j) \in E$ then j is not a s.s., and if $(i, j) \notin E$ then

i is not.

Two-phase algorithm

① we rule out all but one vertices



$n-1$ checks

② we check whether the "last" vertex
is indeed a s.s.

$$\begin{bmatrix} 0 \\ 0 \\ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$2n-2$ checks

Altogether this is an $(3n-2)$ -step algorithm $\rightarrow O(|V|)$

HW. sketch or pseudocode

